
SECTION-2: ARTIFICIAL INTELLIGENCE LAB

Structure	Page No.
2.0 Introduction	41
2.1 Objectives	41
2.2 General Guidelines	42
2.3 Python Libraries for AI	42
2.4 Working with Python	48
2.5 Running Python Programs	50
2.6 Solved Examples of Artificial Intelligence	56
2.7 List of Lab Assignments-Session wise	64
2.8 References/Further Readings	66

2.0 INTRODUCTION

This lab course is designed to have the hands-on experience in AI domain and related algorithms. Further, the AI techniques, algorithms, libraries and frameworks are used to analyse the real-time problems, dataset analysis, interpretations and visualizations.

This course focuses on the implementation aspects of various AI algorithms using Python programming language. For experimentation, Google Collab or ANACONDA, Jupyter Notebook, Eclipse or any relevant platform can be used for execution. Kindly go through the general guidelines and the programming documents carefully before starting experimentations.

This lab course is important to first get the theoretical and mathematical understanding of various algorithms presents in the theory course MCS-081 (Artificial intelligence) and then implement your understanding in Python programming in this lab course, to solve the problems given in the various sessions of this course.

2.1 OBJECTIVES

After completing this lab course you will be able to:

- To develop a strong conceptual understanding of AI, including their scope, significance, and application in solving complex and hard-to-solve problems within a reasonable time by leveraging domain knowledge.
- To familiarize learners with different knowledge representation techniques such as Propositional Logic, First Order Predicate Logic, Rule-based Systems, Semantic Networks, and Frames, and understand how these techniques contribute to building intelligent systems.
- To equip learners with the foundational knowledge and practical insights required to pursue further studies or careers in the rapidly growing fields of Artificial Intelligence and Machine Learning.

2.2 GENERAL GUIDELINES

- It is mandatory to complete all the problems and assignments given in each session of this lab course.
- Lab instructor will be available throughout the lab hours and can support you while working on the exercises. However, you can ask technical difficulties that you encounter related to the algorithm implementation in AI concepts.
- It is strongly recommended that you should add proper comments in the code (preferably before each function, including main function) to give description of function's functionality. The comments should explain the purpose, meaning of the function, and its return value.
- In case, there is need to explain inside the code for better understanding then it should be placed immediately above the relevant function or algorithm's source code.
- The explanation may include where and when the function is necessary in the programming.
- A well-documented program with real input/output data and proper interactivity will be rewarded.
- There should not be any form of plagiarism. For example, if submission from two or more students is found to be the same then none of such submission will be counted. Thus, it is strongly advised that you should not copy somebody else's work.
- It is mandatory to keep the observation book and lab record.
- A separate directory for each student will ensure that no body can read or copy your work.
- In the lab manual, the list of the programs are available. These programs are session wise. These programs are algorithms-based assignments. Thus, it is expected that you must prepare the algorithms and the programs written in the observation book. Lab hours are kept for your experimentation, execution, testing and analysis of the programs.
- It is your responsibility to inform the lab instructor/in-charge after completing the exercise for evaluation. After evaluation, you should get the signature from him/her on the observation book.
- Completed lab assignments should be submitted in the form of a Lab Record in which you have to write the algorithm, program code along with comments and output for various inputs given.
- The total no. of lab sessions (3 hours each) are 20 and the list of assignments is provided session-wise. It is important to observe the deadline given for each assignment.

2.3 PYTHON LIBRARIES FOR AI

Python is a popular programming language for AI, Neural Networks, Time-series Analysis, Mathematical derivations and many more. This programming language is well developed to solve complex problems and prototype development in real-time applications. In Python programming language, there are various operating systems calls, libraries, modules and packages are available which are extensions of other programming languages like C, C++ or

Java. Popular originals use this language in their valuable projects. Python is a high-level, general purpose and interpreted programming language with automatic garbage collection, multi-threading programming paradigms, support for both object-oriented and functional programming, and comprehensive standard library support.

Python is popular in AI domains as well with its great support in emerging fields in computer science. For example, it supports key libraries include NumPy for numerical computations, Pandas for data manipulation, and Matplotlib for data visualization. For building and training deep learning models, Python offers TensorFlow and PyTorch, which are widely used in both academia and industry. In Natural Language Processing (NLP), libraries like NLTK and spaCy provide tools for text processing, while scikit-learn offers extensive support for traditional ML algorithms like regression, clustering, and classification. For reasoning under uncertainty and probabilistic modelling, pgmpy (for probabilistic graphical models) and Pyro (for probabilistic programming) are highly useful. Libraries like SymPy support symbolic reasoning, and tools such as Pyke allow for rule-based inferencing.

- (a) **SymPy:** This library is a powerful python library for symbolic mathematical calculations. Further, it is useful for users to perform algebraic manipulations, calculus, equation solving and many more. This library is mainly used when you deals with formulas, expressions or mathematical objects without replacement of a specific value. For example, this library can perform differentiation, integration, simplification, matrix operations, and solving systems of equations, making it popular in scientific computing, engineering, and research. This library can be integrated with major tools like Jupyter Notebooks, Google Collab and others and it is easy to install and use. This library can develop reasoning with mathematical expressions while handling uncertainties or variable dependencies in symbolic form.
- (b) **Pyke:** This library is useful in rule-based inference engine and can support for forward chaining and backward chaining for logical reasoning development. It is very useful in knowledge-based system development with facts, rules and inferencing mechanisms to solve the problems. For example, problems with reasoning with uncertainty or incomplete information can easily be developed using this library. It uses declarative rule language and Python code for interaction with rules and develop the knowledge bases. Among real-time applications, this library is useful for expert systems, diagnostic systems and other automated reasoning-based systems where encoding the expert knowledge and applying logical inference for conclusions are important. This library is useful to enable other Python libraries that are made for building hybrid systems combining machine learning, rule-based reasoning and probabilistic inference.
- (c) **LogPy:** This library is for logic programming that allows users to perform declarative programming and symbolic reasoning. It is very useful for establishing relations and logic rules to find solutions. The problems which can be efficiently handled using this library include constraint satisfaction, inference engines, knowledge representation and many more. Further, the matching structures unification, backtracking in finding the

solutions, recursive queries, rule-based AI, natural language processing and many more issues can be efficiently handled with this library. It is similar to Prolog for logic development with an addition of seamlessly integration with Python.

- (d) **NetworkX:** This is a powerful Python library designed for initiation, creation, manipulation and analysis of complex networks and graphs. Various types of graphs which are supported using this library include: directed, undirected, multigraphs, modelling of social networks, transportation systems-based graphs, biological networks and representations and many more. This library compute centrality measures, shortest paths, clustering coefficients, community detection and many more. This library is very useful in visualization, interoperability of various data formats, research purposes, data scientists and engineering working in network-based analysis.
- (e) **rdflib:** This library is developed to work with resource description framework data in semantic web. Using this library, users can create, parse, query and serialize the resource description framework with flexible graph-based model. Here, data is represented with triples (subject, predicate, object). In this library, popular resource description framework' format like Turtle, RDF/XML, N-Triples and JSON-LD can use used. Additionally, support for SPARQL, which is standard query language for resource description framework, is available. This allows the developers to effectively search and filter large semantic datasets. This is widely used for linked data applications, knowledge graph development and ontology management.
- (f) **OpenCog:** This is an open-source framework for artificial general intelligence systems. Here, a set of tools, components and algorithms are available for knowledge representation, develop reasoning and data learning. The main core of this library is the AtomSpace which is a flexible graph-based knowledge representation systems to store and manipulate concepts, associated relationships, terminologies and facts. This library also supports natural language processing, machine learning, evolutionary programming and pattern mining. This library is a good tool for advanced AI research and other AI-related tasks which can make it a generalize and reason-based preference across different domains.
- (g) **Scikit-learn:** This is a main machine learning library for simple and efficient data mining and predictive modelling aspects using various tools and features available in this. This library uses NumPy, SciPy and Matplotlib for wide array of algorithms including supervised and unsupervised learning algorithms. Further, it provides support for classification, regression, clustering and dimensionality reduction. Here, user have an option for model selection, preprocessing and evaluation. Thus, it is a comprehensive toolkit for end-to-end machine learning workflows. This library is very common in-use and strong community support made it a go-to library for users including data scientists, developers, students, faculty, researchers and other practitioners.

- (h) **TensorFlow:** This is an open-source library useful for AI/ML-related tasks. Using this library, various tasks like deep learning, neural networks, computer vision, natural language processing and numerical computations can be handled easily. This library provides more flexibility to architectures that allowed models to deploy different platforms such as CPUs, GPUs and TPUs for mobile and edge devices. Further, it support for high-level APIs like Keras for rapid prototype development and improved performance. Using this library lower-level optimization operations for advanced customization can be performed easily. It is also very useful for research and production environments especially research tasks related to model training, evaluation, visualization and deployment. This is a graph-based computational model that provides execution and optimization of large-scale machine learning workflows. Overall, this library is very useful in AI ecosystem.
- (i) **PyTorch:** This is an open-source machine learning library and very useful for deep learning and natural language processing tasks. This library is built using Torch library and very flexible and easy-to-use platform for developing machine learning models. This library is very useful for supports in dynamic, computational graphs, developers to modify the network behaviours, and complex model building and debugging. Using this library, it is easy to perform tasks like image classification, computer vision, natural language processing and reinforcement learning. This library can easy be integrated with NumPy, SciPy and Cython. With this integration, it is easy to perform seamless data processing and model deployment. Overall, this library, because of its features, performance and flexibility, is widely used for academic research and industrial applications.
- (j) **Keras:** This is an open-source, high-level neural network API. This library can be used for building and training deep learning models. This library is simple, easy to use, modular and helpful for sequential and functional APIs especially in field of deep learning models. This library has wide range of pre-trained models and layers. As a result, tasks like image classification, object detection and text processing can be simplified. This library is very useful in quick prototyping, model development and processing. Library has the support for backends like TensorFlow, Theano, and Microsoft Cognitive Toolkit (CNTK). In GUP and TPU processing, this library is also useful for making efficient large-scale machine learning tasks.
- (k) **Pandas:** This python library is an open-source library for data manipulation and analysis. This library is very useful for handling structured data. The library provides a powerful data structures and functions in data cleaning, transformation and analysis tasks which make the machine learning much easier for real-time applications. This library can presents the large datasets in tabular form like SQL tables or excel spreadsheets does. There are various other tasks and functionalities which this library can perform like data indexing, reshaping, merging and aggregation. All of these tasks are important for data scientists and analysts in preparing data machine learning models or visualization. Thus,

in data science ecosystem, this library can handle missing data, time series data analysis and complex data operations.

- (l) **NumPy:** This library can do numerical computing for large multi-dimensional arrays and matrices. The library is high-performance multidimensional array object called 'ndarray'. Using this library, a comprehensive set of mathematical functions and operations for arrays can be conducted. As a result, this library can do extensive scientific computations, data analysis and machine learning applications. The important areas where these tasks can be conducted include linear algebra, Fourier transformations, random number generation and statistical computations. Like any other useful library, this library can be integrated with other libraries like Pandas, Matplotlib, TensorFlow, and SciPy. Overall, data processing and mathematical computations would be much easier with this library.
- (m) **Matplotlib:** This data visualization library are useful in creating static, animated, and interactive visualizations. Using this library, tasks like 2D plots with line charts, bar graphs, histograms, scatter plots and pie charts can be drawn easily. In addition to 2D plots, this library can be useful for 3D plotting as well and capabilities. Using this library, it is easy to do high-level plotting functions. For example, 'pyplot' module in this library allows users to customize every aspect of a plot. In this plot, labels, legends, colour maps and grid lines can be easily drawn. Using datasets and available libraries, it is easy to perform data analysis, machine learning and scientific computing in visualization. Using Matplotlib library, integration with other libraries like Pandas and NumPy will make this a versatile tools for data science and research activities.
- (n) **Seaborn:** This library is important visualization library built on top of Matplotlib. This library is helpful in creation of aesthetically pleasing and informative statistical graphics. For example, heatmaps, violin plots, box plots, pair plots, categorical plots can be easily drawn using this library as it provides a high-level interface for drawing. To visualize the complex relationships in datasets, this library support through various built-in themes and colour palettes. With Pandas data structure integration, plotting of DataFrames Seaborn is very easy. This library is very useful in data analysis and machine learning tasks as well. The importance of this library in these tasks mainly include the exploration and visualization of data patterns effectively.
- (o) **AIMA-Python:** This library is an open-source library for implementing algorithms and examples. The algorithms and examples which are mainly using this library include search algorithms, game playing, knowledge representation, probabilistic reasoning, machine learning and reinforcement learning. Students, researchers and practitioners can easily take advantage of this library in understanding and experimenting with AI concepts mainly through Python codes. This library is easy to understand in terms of implementation and building AI algorithms. Here, real world applications can be easily taken up for learning AI algorithms and their analysis.

- (p) **SciPy:** This library is an open-source software for mathematics, science and engineering. This library uses NumPy and provides a collection of mathematical examples, algorithms, functions and tools for scientific and technical computations. In this library, there are modules available for optimization, linear algebra, integration, interpolation, signal processing, image processing, statistical computations and various other analysis. This library can be used for data science, machine learning and scientific research. In these tasks, this library mainly performs complex mathematical computations and handling of large datasets. This library is flexible enough to handle robust scientific and numerical tasks which researchers and engineers have to perform in solving mathematical problems and analysis.
- (q) **NLTK:** This python library is very powerful and widely used. This library can be used with human language data and provide easy to use interface for over 50 corpora and lexical resources. The major tasks such as WordNet, includes text processing for tokenization, stemming, tagging, parsing and semantic reasoning. This library is widely used for natural language processing applications which enable the important tasks like sentiment analysis, text classification, language modelling and text analysis. As it has wide use in real-time applications, the students, researchers and other IT professionals use this library at large for prototyping and experimenting with language-based tasks.
- (r) **spaCy:** This is another advanced and efficient python library for natural language processing tasks. This library was mainly developed for production use, give fast performance and easy to use. Like other natural language processing libraries, this library can also perform tokenization, entity recognition, part of speech tagging, dependency parsing and word vectors. This library is widely used for industrial natural language processing application as it provides high-speed processing capabilities and pre-trained models for multiple languages. This library support deep learning workflows and can be integrated with popular frameworks like TensorFlow and PyTorch. All of these functionalities make this library an ideal choice for building scalable natural language processing applications.
- (s) **OpenAI Gym:** This library is popularly used for OpenAI-based reinforcement learning algorithms and associated tasks. This library can be widely used for pre-built environments like classic control problems, Atari games, and simulated robotics tasks. These tasks help the researchers and developers to test and train reinforcement learning models. OpenAI Gym library simplifies the process of design, test and benchmarking reinforcement learning algorithms. Here, there is use of standard API and a large suite of challenging environments. All of these features make this library an efficient tool for researchers, academia and industry applications. With this library, intelligent agents can be build that are capable of learning from interactions with the environment.
- (t) **Stable-Baselines3:** This library is a popular and reliable library built on top of PyTorch in developing and training reinforcement learning agents. Using this library, one can easily make a clean, efficient and modular

implementation of important reinforcement learning algorithms. For example, Deep Q-Learning (DQN), Proximal Policy Optimization (PPO), Advantage Actor-Critic (A2C), and Soft Actor-Critic (SAC) can easily be practiced using this library. This library is developed for an ease of use, rapid experimentation and reproducibility in reinforcement learning research and applications. This library also support OpenAI Gym environments which can be easily integrated with custom environments and make a preferred choice for both academia and industry-based real-time applications and projects.

- (u) **PySC2:** This is a Python-based reinforcement learning library and also known as Python StarCraft II Learning Environment. This library is developed for extensive research in complex strategic game environments. For example, this library provides an interface to the popular real-time strategy game StarCraft II. Thus, it is easy to train and evaluate reinforcement learning agents using this library especially for a challenging multi-agent settings. This library exposes a rich and complex environment with good observability, long-term planning and multi-agent interactions. All of these features make this library more useful and ideal for advancing the reinforcement learning algorithms especially in real-world scenarios.
- (v) **Dopamine:** This is another library useful for reinforcement learning algorithms and associated tasks. This library is a lightweight and flexible Python library for designing and evaluating important algorithms focused on deep reinforcement learning. This library is easy-to-use in terms of its implementation in standard reinforcement learning algorithms such as Deep Q-Learning or its variants. This library is very much in real-time use and allows the researchers or professional to experiment quickly and iteratively with new ideas. As a result, this library provides reproducibility, simplicity and benchmarking. This library provides support for seamlessly integration with environments like OpenAI Gym and Atari 2600 games.
- (w) **Feature-engine:** This is an open-source Python library designed for engineering jobs in machine learning pipelines. This library provides a set of tools as well which can be used to transform and engineer new features after analysing raw data, applying encoding categorical variables, handling missing data, innovating creative and interactive terms and conducting scalable numerical features. This library is designed to work seamlessly with the scikit-learn library and ensure various feature engineering steps. These steps are useful in machine learning pipelines which is the main purpose of this library. This library can be useful in improving the performance with high quality input features, creativity and managing the tasks in an organized and reproducible way.

2.4 WORKING WITH PYTHON

The important ways of using Python for Artificial Intelligence are described as follow:

Method 1: Installing Python Locally

Using this method, Python is downloaded and installed from official Python website and then used directly via a terminal or command prompt. Further, an IDE (like Eclipse, PyCharm, VS Code or Jupyter Notebook) can also be used for experimentation.

Steps:

1. Go to website : <https://www.python.org>
2. Download and install the latest Python version for use.
3. Install libraries using command like: `pip install numpy pandas scikit-learn matplotlib seaborn tensorflow keras`
4. Run Python code in text editors like: VS Code, PyCharm, Sublime Text, etc.

There are various advantages and disadvantages of this approach. The major advantages include: (i) This installation provides full control over the environment and your configurations. (ii) Python scripts can run independently. (iii) This method is preferred approach for large and advanced Machine Learning Projects.

Method 2: Using Anaconda with Jupyter Notebook

In this method, Anaconda is installed first followed by data science platform that comes pre-installed with Jupyter Notebook for experimentation. This method can be preferably used for Machine Learning and Data Science tasks.

Steps:

1. Goto website: <https://www.anaconda.com>
2. Download and install the most suitable version for your operating system.
3. Open the Jupyter Notebook using following command: `jupyter notebook`
4. Write and run Machine Learning code:
 - a. After opening the Jupyter Notebook, libraries like pandas, sklearn, etc. can be installed and coding for problem statements can be started.

This method also has various advantages and disadvantages. The major advantages include: (i) It is an easy-to-use method with web-based interface. (ii) Pre-installed libraries and environment parameters save time. (iii) This method is preferred for prototyping, data analysis and Machine Learning model development. The major disadvantages of this method are: (i) it takes 2 to 3 GB of space for installation i.e. installation is slightly heavier. (ii) This method may arise dependency conflicts.

Method 3: Using Google Collab (an online method)

This is an online cloud-based platform for machine learning practice. This is developed by Google and a free Jupyter Notebook environment that runs entirely on Google Servers.

Steps:

1. **Goto Website:** <https://colab.research.google.com>
2. Click on New Notebook
3. Import libraries in cell like:

```
import pandas as pd
```

```
import numpy as np
import sklearn
```

4. Execute it.
5. You can select runtime GPU/TPU for leveraging the hardware acceleration.

This method also has various advantages and disadvantages. For example, (i) No installation required in this method. (ii) it provides free access to GPUs/TPUs for faster model training. (iii) It is an easy platform for collaboration and sharing notebooks. The major disadvantages include: (i) this method required active internet connection to work, and (ii) It provides limited compute time per session.

Method 4: Using Kaggle Kernels (an online method)

This method is similar to previous method (i.e. Google Colab). This method uses Kaggle for online practice. This provides an opportunity to directly integrates with datasets from Kaggle competitions and supports from Jupyter Notebook-style coding.

Steps:

1. Goto website: <https://www.kaggle.com/kernels>
2. Create a New Kernel/Notebook, and execute your code.

The major advantages of this method are: (i) This platform provides access to thousands of public datasets. (ii) It has pre-installed libraries and tools for practice. (iii) It provides support for GPU/TPU like Colab. The major disadvantages of this method are: It requires an active internet connection, and (ii) It has limited compute resources.

Method 5: Using Docker Containers

This method support the use of docker for creating isolated environments for different machine learning projects. Thus, it is very suitable for large-scale or production-level deployments.

Steps:

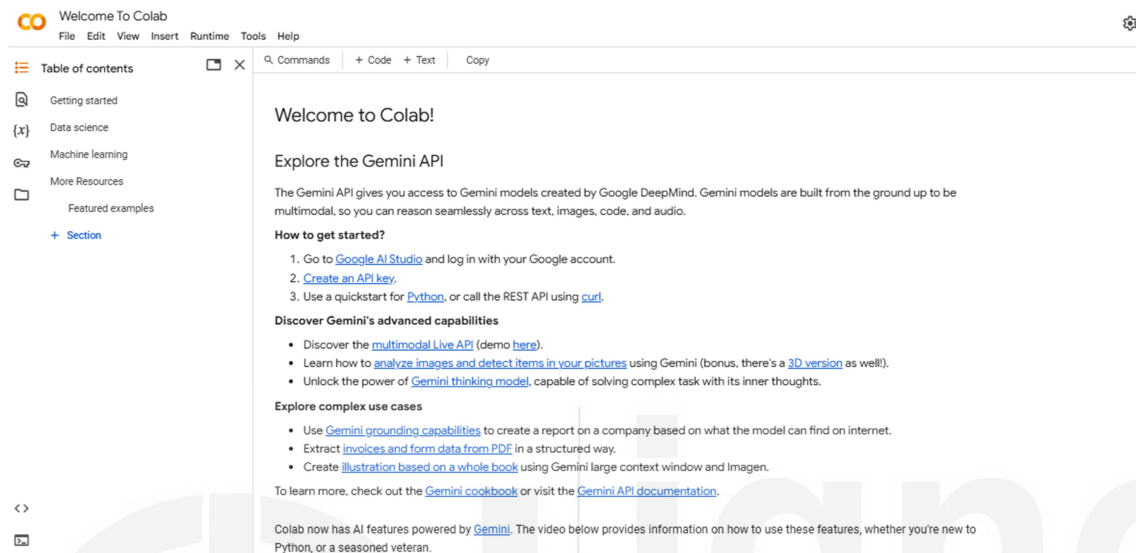
1. Goto website: <https://www.docker.com>
2. Download Docker and install it.
3. Pull Machine Learning Containers like:
docker pull jupyter/scipy-notebook
4. Run the container like:
docker run -p 8888:8888 jupyter/scipy-notebook
5. Access the Jupyter Notebook in the browser using URL:
<http://localhost:8888>

This method has various advantages like: (i) Provides complete isolation of the environment, (ii) It is comparatively use to use compare to other methods, (iii) This method is preferred method for deploying models in production. The major disadvantages of this methods are: (i) This is a resource-intensive methods, and (ii) It give higher learning curve.

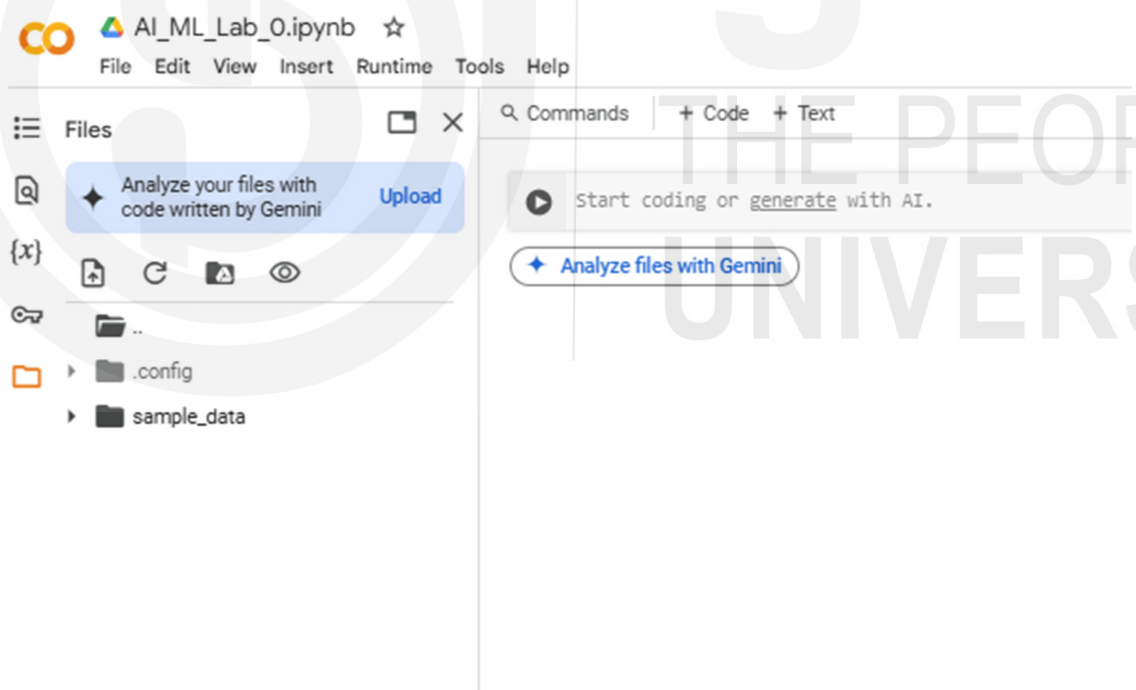
2.5 RUNNING PYTHON PROGRAMS

Follow are the stating steps and example for running python programs n this lab:

Step 1: In google Colab, click file option and select new workbook.



Step 2: A new Jupyter Notebook will open where you can enter your code and execute it.



Step 3: This step demonstrates an example and its execution in Google Colab.

Problem: Consider you are employee of a petroleum industry-based organization, and your task is to predict the production rate of oil wells, optimizing extraction process and manage resources efficiently. As this sector is very sensitive, it is very hard to collect real-world data. Thus, your role is to first generate synthetic data for simulating oil well production rate based on

various factors like reservoir pressure, well depth and water cut. Write python program to build AI/ML model for predicting production rates and analysing the impact of various factors.

(Note: This problem is in-depth AI/ML-based problem for you at this beginning state. Aim of you is to just learn the use of Google Colab, synthetic data generation, python-based code execution in Google Colab, and observe the outputs).

Step 3a: Import libraries

```
✓  # Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
```

Step 1: Enter your code in cell.

Step 2: Click here to execute.

Step 3b: Generate and display Synthetic data.

```
✓  np.random.seed(42)

# Number of samples
n_samples = 1000

# Synthetic features
reservoir_pressure = np.random.uniform(2000, 5000, n_samples) # psi
well_depth = np.random.uniform(5000, 15000, n_samples) # feet
water_cut = np.random.uniform(0.1, 0.7, n_samples) # percentage

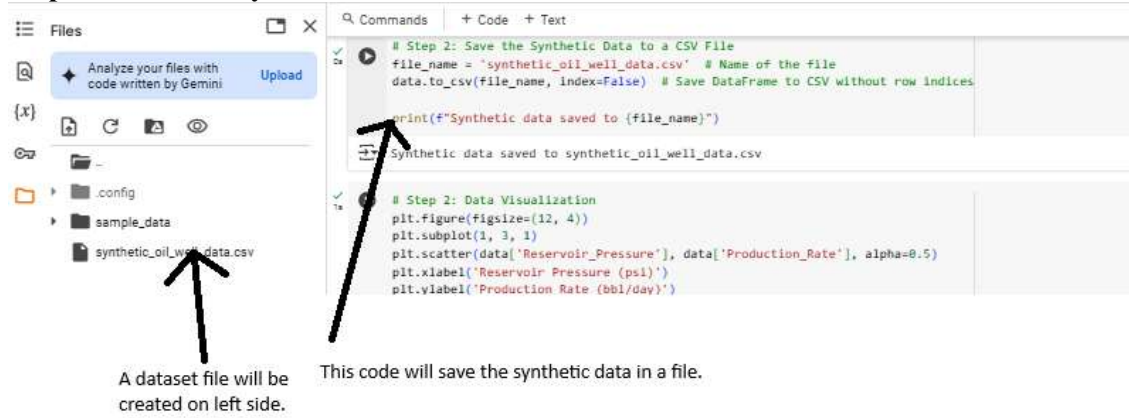
# Synthetic target: Production rate (barrels per day)
production_rate = (
    1000 * (reservoir_pressure / 5000) +
    500 * (well_depth / 15000) -
    2000 * water_cut +
    np.random.normal(0, 100, n_samples) ) # Adding noise

# Create a DataFrame
data = pd.DataFrame({
    'Reservoir_Pressure': reservoir_pressure,
    'Well_Depth': well_depth,
    'Water_Cut': water_cut,
    'Production_Rate': production_rate
})

# Display the first few rows of the dataset
print(data.head())
```

	Reservoir_Pressure	Well_Depth	Water_Cut	Production_Rate
0	3123.620357	6851.329288	0.257023	445.048532
1	4852.142919	10419.009474	0.248187	883.054936
2	4195.981825	13729.458359	0.643753	77.696413
3	3795.975453	12322.248864	0.249728	533.885708
4	2468.055921	13065.611479	0.263170	523.986294

Step 3c: Save the Synthetic data in a file.



The screenshot shows a Jupyter Notebook interface. On the left, the 'Files' pane displays a directory structure with files like '.config', 'sample_data', and 'synthetic_oil_well_data.csv'. An arrow points from the text 'A dataset file will be created on left side.' to the 'synthetic_oil_well_data.csv' file. In the main code area, two code cells are visible. The first cell contains code to save synthetic data to a CSV file, with an arrow pointing from the text 'This code will save the synthetic data in a file.' to it. The second cell contains code for data visualization.

```
# Step 2: Save the Synthetic Data to a CSV File
file_name = 'synthetic_oil_well_data.csv' # Name of the file
data.to_csv(file_name, index=False) # Save DataFrame to CSV without row indices

print(f"Synthetic data saved to {file_name}")

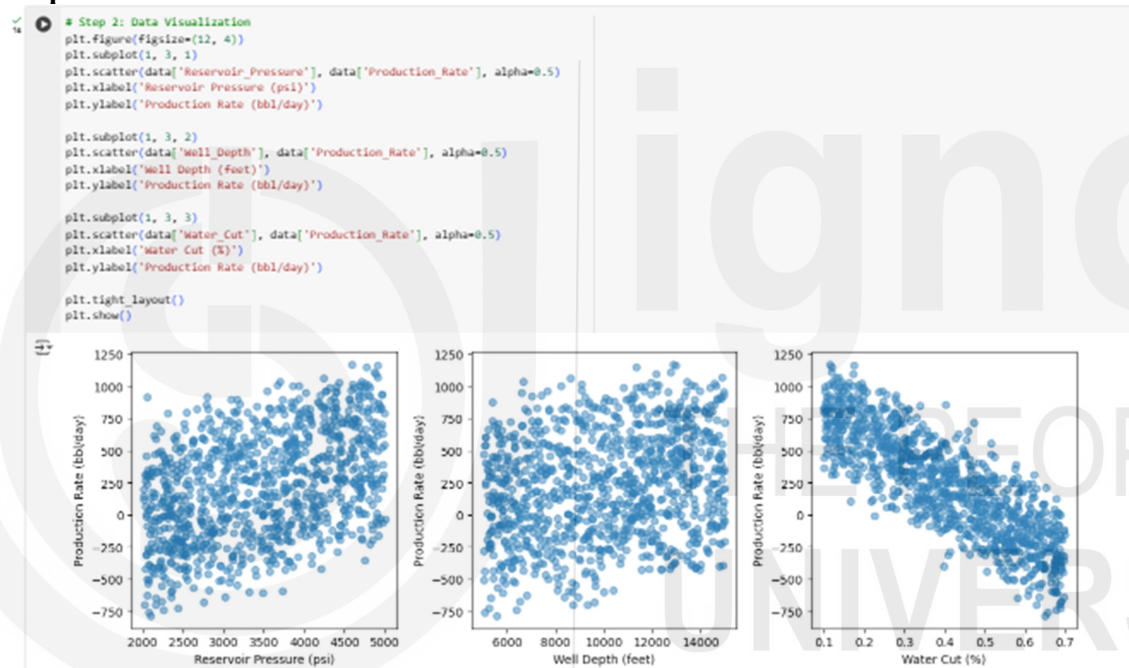
Synthetic data saved to synthetic_oil_well_data.csv

# Step 2: Data Visualization
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.scatter(data['Reservoir_Pressure'], data['Production_Rate'], alpha=0.5)
plt.xlabel('Reservoir Pressure (psi)')
plt.ylabel('Production Rate (bbl/day)')
```

A dataset file will be created on left side.

This code will save the synthetic data in a file.

Step 3d: Visualize the data.



Step 3e: Preparing data for ML Model

```
# Step 3: Prepare Data for ML Model
X = data[['Reservoir_Pressure', 'Well_Depth', 'Water_Cut']]
y = data['Production_Rate']
```

Step 3f: Prepare the data into training and testing sets.

```
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Step 3g: Train the data using Random Forest Regressor

```
# Step 4: Train a Random Forest Regressor
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

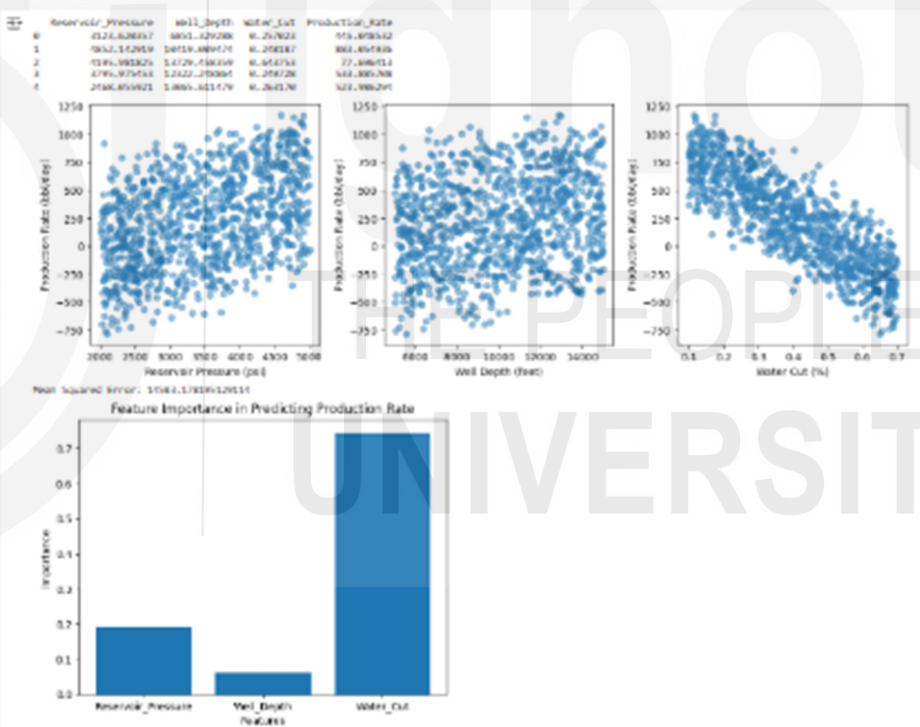
Step 3h: Evaluate the Model

```
# Step 5: Evaluate the Model
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
```

```
Mean Squared Error: 14583.178195129114
```

Step 3i: Perform Feature Analysis

```
# Step 6: Feature Importance Analysis
feature_importances_ = model.feature_importances_
features = X.columns
plt.bar(features, feature_importances_)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Feature Importance in Predicting Production Rate")
plt.show()
```



Following is the complete code:

```
# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
```

```
# Step 1: Generate Synthetic Data
```

```

np.random.seed(42)

# Number of samples
n_samples = 1000

# Synthetic features
reservoir_pressure = np.random.uniform(2000, 5000,
n_samples) # psi
well_depth = np.random.uniform(5000, 15000, n_samples) #
feet
water_cut = np.random.uniform(0.1, 0.7, n_samples) #
percentage

# Synthetic target: Production rate (barrels per day)
production_rate = (
    1000 * (reservoir_pressure / 5000) +
    500 * (well_depth / 15000) -
    2000 * water_cut +
    np.random.normal(0, 100, n_samples) # Adding noise

# Create a DataFrame
data = pd.DataFrame({
    'Reservoir_Pressure': reservoir_pressure,
    'Well_Depth': well_depth,
    'Water_Cut': water_cut,
    'Production_Rate': production_rate
})

# Display the first few rows of the dataset
print(data.head())

# Step 2: Data Visualization
plt.figure(figsize=(12, 4))
plt.subplot(1, 3, 1)
plt.scatter(data['Reservoir_Pressure'], data['Production_Rate'],
alpha=0.5)
plt.xlabel('Reservoir Pressure (psi)')
plt.ylabel('Production Rate (bbl/day)')

plt.subplot(1, 3, 2)
plt.scatter(data['Well_Depth'], data['Production_Rate'],
alpha=0.5)
plt.xlabel('Well Depth (feet)')
plt.ylabel('Production Rate (bbl/day)')

plt.subplot(1, 3, 3)
plt.scatter(data['Water_Cut'], data['Production_Rate'],
alpha=0.5)
plt.xlabel('Water Cut (%)')
plt.ylabel('Production Rate (bbl/day)')

plt.tight_layout()

```

```

plt.show()

# Step 3: Prepare Data for ML Model
X = data[['Reservoir_Pressure', 'Well_Depth', 'Water_Cut']]
y = data['Production_Rate']

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Step 4: Train a Random Forest Regressor
model = RandomForestRegressor(n_estimators=100,
random_state=42)
model.fit(X_train, y_train)

# Step 5: Evaluate the Model
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print(f'Mean Squared Error: {mse}')

# Step 6: Feature Importance Analysis
feature_importances = model.feature_importances_
features = X.columns
plt.bar(features, feature_importances)
plt.xlabel('Features')
plt.ylabel('Importance')
plt.title('Feature Importance in Predicting Production Rate')
plt.show()

```

2.6 SOLVED EXAMPLES OF ARTIFICIAL INTELLIGENCE

In the following section we will study few of the solve problems related to artificial Intelligence.

1. Write a Python Program to implement Alpha-Beta Pruning Algorithm

Solution:

```

import math
def alpha_beta_pruning(node_index, depth, is_maximizing_player, values,
alpha, beta):
    # Base condition: if leaf node is reached
    if depth == 0:
        return values[node_index]
    if is_maximizing_player:
        max_eval = -math.inf
        for i in range(2): # two children for each node
            eval = alpha_beta_pruning(2 * node_index + i, depth - 1, False,
values, alpha, beta)
            max_eval = max(max_eval, eval)
        alpha = max(alpha, eval)

```

```

        if beta <= alpha:
            break #  $\beta$  cut-off
        return max_eval
    else:
        min_eval = math.inf
        for i in range(2): # two children for each node
            eval = alpha_beta_pruning(2 * node_index + i, depth - 1, True,
values, alpha, beta)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break #  $\alpha$  cut-off
        return min_eval
# Example usage
if __name__ == "__main__":
    # Sample leaf nodes for a binary tree of depth 3 (i.e.,  $2^3 = 8$  leaf nodes)
    values = [3, 5, 6, 9, 1, 2, 0, -1]
    depth = 3 # height of the tree
    alpha = -math.inf
    beta = math.inf
    optimal_value = alpha_beta_pruning(0, depth, True, values, alpha, beta)
    print("The optimal value is:", optimal_value)
Output:

```

```

import math

# Define the Alpha-Beta pruning function
def alpha_beta_pruning(node_index, depth, is_maximizing_player, values, alpha, beta):
    # Base condition: if leaf node is reached
    if depth == 0:
        return values[node_index]

    if is_maximizing_player:
        max_eval = -math.inf
        for i in range(2): # two children for each node
            eval = alpha_beta_pruning(2 * node_index + i, depth - 1, False, values, alpha, beta)
            max_eval = max(max_eval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha:
                break #  $\beta$  cut-off
        return max_eval
    else:
        min_eval = math.inf
        for i in range(2): # two children for each node
            eval = alpha_beta_pruning(2 * node_index + i, depth - 1, True, values, alpha, beta)
            min_eval = min(min_eval, eval)
            beta = min(beta, eval)
            if beta <= alpha:
                break #  $\alpha$  cut-off
        return min_eval

# Example usage
if __name__ == "__main__":
    # Sample leaf nodes for a binary tree of depth 3 (i.e.,  $2^3 = 8$  leaf nodes)
    values = [3, 5, 6, 9, 1, 2, 0, -1]
    depth = 3 # height of the tree
    alpha = -math.inf
    beta = math.inf

    optimal_value = alpha_beta_pruning(0, depth, True, values, alpha, beta)
    print("The optimal value is:", optimal_value)

```

The optimal value is: 5

2. Write a Python Program to implement Iterative Deepening Depth First search (IDDFS)

Solution:

```
# Depth-Limited DFS
def DLS(graph, node, target, limit):
    print(f"Visiting Node: {node} at depth {limit}")
    if node == target:
        return True
    if limit <= 0:
        return False
    for neighbor in graph.get(node, []):
        if DLS(graph, neighbor, target, limit - 1):
            return True
    return False

# Iterative Deepening DFS
def IDDFS(graph, start, target, max_depth):
    for depth in range(max_depth + 1):
        print(f"\nSearching at depth level: {depth}")
        if DLS(graph, start, target, depth):
            print(f"Target node '{target}' found at depth {depth}")
            return True
    print(f"Target node '{target}' not found within depth {max_depth}")
    return False

# Example graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': [],
    'G': []
}

# Example usage
if __name__ == "__main__":
    start_node = 'A'
    target_node = 'G'
    max_depth = 3
    IDDFS(graph, start_node, target_node, max_depth)
```

```

# Depth-limited DFS
def DLS(graph, node, target, limit):
    print(f"Visiting Node: {node} at depth {limit}")
    if node == target:
        return True
    if limit <= 0:
        return False
    for neighbor in graph.get(node, []):
        if DLS(graph, neighbor, target, limit - 1):
            return True
    return False

# Iterative Deepening DFS
def IDDFS(graph, start, target, max_depth):
    for depth in range(max_depth + 1):
        print(f"Searching at depth level: {depth}")
        if DLS(graph, start, target, depth):
            print(f"Target node '{target}' found at depth {depth}")
            return True
    print(f"Target node '{target}' not found within depth {max_depth}")
    return False

# Example graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': [],
    'G': []
}

# Example usage
if __name__ == "__main__":
    start_node = 'A'
    target_node = 'G'
    max_depth = 3
    IDDFS(graph, start_node, target_node, max_depth)

```

```

Searching at depth level: 0
Visiting Node: A at depth 0

Searching at depth level: 1
Visiting Node: A at depth 1
Visiting Node: B at depth 0
Visiting Node: C at depth 0

Searching at depth level: 2
Visiting Node: A at depth 2
Visiting Node: B at depth 1
Visiting Node: D at depth 0
Visiting Node: E at depth 0
Visiting Node: C at depth 1
Visiting Node: F at depth 0

Searching at depth level: 3
Visiting Node: A at depth 3
Visiting Node: B at depth 2
Visiting Node: D at depth 1
Visiting Node: E at depth 1
Visiting Node: G at depth 0
Target node 'G' found at depth 3

```

3. Write a Python Program to implement Best First Search algorithm

Solution:

```
import heapq

def best_first_search(graph, heuristics, start, goal):
    visited = set()
    priority_queue = []

    # Push the start node into the priority queue with its heuristic value
    heapq.heappush(priority_queue, (heuristics[start], start))

    while priority_queue:
        current_cost, current_node = heapq.heappop(priority_queue)
        print(f"Visiting Node: {current_node}")

        if current_node == goal:
            print(f"Goal node '{goal}' found!")
            return True

        if current_node not in visited:
            visited.add(current_node)

            for neighbor in graph.get(current_node, []):
                if neighbor not in visited:
                    heapq.heappush(priority_queue, (heuristics[neighbor],
neighbor))

    print(f"Goal node '{goal}' not reachable from '{start}'")
    return False

# Example graph (adjacency list)
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': [],
    'G': []
}

# Heuristic values for each node (lower is better)
heuristics = {
    'A': 5,
    'B': 3,
    'C': 4,
    'D': 6,
    'E': 2,
    'F': 6,
    'G': 1 # Goal node should have the lowest heuristic ideally
}
```

```
# Example usage
if __name__ == "__main__":
    start_node = 'A'
    goal_node = 'G'
    best_first_search(graph, heuristics, start_node, goal_node)
```

Output:

```
➡ Visiting Node: A
   Visiting Node: B
   Visiting Node: E
   Visiting Node: G
   Goal node 'G' found!
```

4. Write a Python Program to implement A* Algorithm

Solution:

```
import heapq

def a_star_search(graph, heuristics, start, goal):
    open_list = []
    heapq.heappush(open_list, (heuristics[start], 0, start, [start])) # (f = g + h,
g, node, path)
    closed_set = set()

    while open_list:
        f, g, current, path = heapq.heappop(open_list)
        print(f'Visiting Node: {current} with f = {f}, g = {g}, h =
{heuristics[current]}')

        if current == goal:
            print(f'Goal node '{goal}' found! Path: {' -> '.join(path)}")
            return path

        if current in closed_set:
            continue
        closed_set.add(current)

        for neighbor, cost in graph.get(current, []):
            if neighbor not in closed_set:
                g_new = g + cost
                f_new = g_new + heuristics[neighbor]
                heapq.heappush(open_list, (f_new, g_new, neighbor, path +
[neighbor]))

    print(f'Goal node '{goal}' not reachable from '{start}''")
    return None

# Example weighted graph as adjacency list (node: [(neighbor, cost), ...])
graph = {
```

```

'A': [('B', 1), ('C', 3)],
'B': [('D', 1), ('E', 3)],
'C': [('F', 5)],
'D': [],
'E': [('G', 2)],
'F': [],
'G': []
    }

```

Heuristic values for nodes

```

heuristics = {
    'A': 6,
    'B': 4,
    'C': 5,
    'D': 2,
    'E': 3,
    'F': 6,
    'G': 0 # Goal node
}

```

Example usage

```

if __name__ == "__main__":
    start_node = 'A'
    goal_node = 'G'
    a_star_search(graph, heuristics, start_node, goal_node)

```

Output:

```

➡ Visiting Node: A with f = 6, g = 0, h = 6
Visiting Node: B with f = 5, g = 1, h = 4
Visiting Node: D with f = 4, g = 2, h = 2
Visiting Node: E with f = 7, g = 4, h = 3
Visiting Node: G with f = 6, g = 6, h = 0
Goal node 'G' found! Path: A -> B -> E -> G

```

5. Write a Python Program to implement AO* Algorithm

Solution:

```

class Node:
    def __init__(self, name):
        self.name = name
        self.children = [] # List of tuples: (child_nodes, cost)
        self.heuristic = 0
        self.solved = False
        self.best_child = None

def ao_star(node, graph, solved_nodes):
    if node.solved:
        return node.heuristic

    print(f'Expanding Node: {node.name}')

    min_cost = float('inf')

```

```

best_option = None

for child_group, cost in graph.get(node.name, []):
    total_cost = cost
    for child_name in child_group:
        total_cost += nodes[child_name].heuristic

    if total_cost < min_cost:
        min_cost = total_cost
        best_option = (child_group, cost)

if best_option is None:
    node.solved = True
    return node.heuristic

node.best_child = best_option[0]
node.heuristic = min_cost

all_solved = True
for child_name in best_option[0]:
    child_node = nodes[child_name]
    ao_star(child_node, graph, solved_nodes)
    if not child_node.solved:
        all_solved = False

if all_solved:
    node.solved = True
    solved_nodes.add(node.name)

return node.heuristic

def print_solution(node):
    if node.solved:
        print(f'{node.name}', end='')
        if node.best_child:
            print(" -> (", end='')
            for i, child in enumerate(node.best_child):
                print(child, end='')
                if i != len(node.best_child) - 1:
                    print(" AND ", end='')
            print(")")
            for child in node.best_child:
                print_solution(nodes[child])

# Define graph structure
# Format: node: [(child_nodes), cost), ...]
graph = {
    'A': [ ('B', 'C'], 1), ('D'], 2) ],
    'B': [ ('E'], 1), ('F'], 2) ],
    'C': [ ('G'], 2) ],
    'D': [],
    'E': [],

```

```

    'F': [],
    'G': []
}

# Create nodes and heuristics
nodes = {
    'A': Node('A'),
    'B': Node('B'),
    'C': Node('C'),
    'D': Node('D'),
    'E': Node('E'),
    'F': Node('F'),
    'G': Node('G')
}

# Assign heuristic values (example)
nodes['A'].heuristic = 10
nodes['B'].heuristic = 4
nodes['C'].heuristic = 2
nodes['D'].heuristic = 6
nodes['E'].heuristic = 3
nodes['F'].heuristic = 2
nodes['G'].heuristic = 0

# Solve the graph
solved_nodes = set()
ao_star(nodes['A'], graph, solved_nodes)

# Print the solution
print("\nAO* Solution Path:")
print_solution(nodes['A'])

```

Output:

```

➡ Expanding Node: A
Expanding Node: B
Expanding Node: E
Expanding Node: C
Expanding Node: G

AO* Solution Path:
A -> (B AND C)
B -> (E)
EC -> (G)
G

```

2.7 LIST OF LAB ASSIGNMENTS-SESSION WISE

Now, let us try solving a different kind of a AI and Machine Learning problems using Python.

2.8 LAB SESSIONS ON AI

In each of the problems given in the respective sessions, you are required to write the algorithm for the given problem, and map the steps of your algorithm with your python code, by using suitable comments.

Session 1:

- a) Write a Python Program to Solve N-Queen Problem without using Recursion.
- b) Write a Python Program to implement the Backtracking approach to solve N Queen's problem

Session 2:

- a) Write a Python Program to implement Min-Max Algorithm
- b) Write a Python Program to implement Breadth First Search
- c) Write a Python program to simulate a simple stimulus-response agent that reacts to input based on predefined rules.
- d) Write a Python program to Train a simple machine learning model (linear regression) to predict a value based on input data.

Session 3:

- a) Write a Python Program to implement Depth First Search
- b) Write a Python Program to implement Iterative Deepening DepthFirst search (IDDFS)

Session 4:

Write a Python Program to implement AO* Algorithm

Session 5:

Write a Python Program to implement for IDA* (Iterative Deepening A*) algorithm

Session 6:

You are given a 3x3 grid with 8 tiles and one empty space. The tiles are numbered from 1 to 8, and the goal is to rearrange the tiles from a given initial state to a goal state by sliding the tiles into the empty space. Use a search algorithm (e.g., Breadth-First Search or A*) and write a python program to find the sequence of moves required to solve the puzzle.

Session 7:

You are given a graph representing a road network with nodes as locations and edges as roads connecting them. Each edge has a uniform cost. Implement **Bidirectional Search using python** to find the shortest path between a start node and a goal node.

Session 8:

You are given a propositional logic formula, and your task is to determine whether it is **valid** (always true), **satisfiable** (true for some assignment of variables), or **unsatisfiable** (never true). For example, consider the formula $(P \vee Q) \wedge (\neg P \vee \neg Q)$. Write a Python program that takes such a

formula as input, generates all possible truth assignments for the variables, and evaluates the formula to determine its validity and satisfiability. The program should output whether the formula is valid, satisfiable, or unsatisfiable.

Session 9:

You are given a set of FOL clauses, and your task is to use **resolution with variables** to determine whether the set is **satisfiable** or **unsatisfiable**. For example, consider the clauses: $\neg P(x) \vee Q(x)$, $\neg Q(y) \vee R(y)$, $P(a)$, and $\neg R(b)$. Write a Python program that applies the resolution rule iteratively, using unification to resolve clauses with variables, and checks whether the empty clause (indicating unsatisfiability) can be derived. The program should output whether the set of clauses is satisfiable or unsatisfiable.

Session 10:

You are given a knowledge base (KB) of FOL sentences and a query. Your task is to determine whether the KB **entails** the query, i.e., whether the query is true in all models where the KB is true. For example, consider the KB: $\forall x (P(x) \rightarrow Q(x))$ and $P(a)$, and the query $Q(a)$. Write a Python program that checks whether the KB entails the query by constructing a proof or using a truth table approach. The program should output whether the KB entails the query.

2.8 REFERENCES/FURTHER READINGS

- [1] K. F. Improve, “AI with Python tutorial,” *GeeksforGeeks*, 24-Jan-2024. [Online]. Available: <https://www.geeksforgeeks.org/python-ai/>. [Accessed: 12-Mar-2025].
- [2] *Udemy.com*. [Online]. Available: https://www.udemy.com/course/the-introduction-of-ai-and-machine-learning-with-python/?srsltid=AfmBOopPjmU_gX9iyaBEJJ6hPmKTdZPPV3_1t1a9Jo0i-2mq19UIId2F&couponCode=NVDIN35. [Accessed: 12-Mar-2025].
- [3] H. M. Elsherif, *Python for artificial intelligence: A comprehensive guide*. Eldonusa, 2024.
- [4] T. Carter, *Artificial intelligence with python: Building intelligent applications*. Independently Published, 2024.
- [5] A. Artasanchez and P. Joshi, *Artificial Intelligence with Python: Your complete guide to building intelligent apps using Python 3.x, 2nd Edition*, 2nd ed. Birmingham, England: Packt Publishing, 2020.
- [6] P. Xiao, *Artificial intelligence programming with python: From zero to hero*. Nashville, TN: John Wiley & Sons, 2022.
- [7] P. Fagundes, *Introduction to artificial intelligence with python: A practical learning journey*. Independently Published, 2024.
- [8] T. T. Teoh and Z. Rong, *Artificial intelligence with python*, 1st ed. Singapore,

Singapore: Springer, 2023.

- [9] S. Lynch, *Python for scientific computing and artificial intelligence*. Philadelphia, PA: Chapman & Hall/CRC, 2023.
- [10] I. Pjp, *Learn all about Artificial intelligence using Python*. Independently Published, 2023.
- [11] P. Gupta and A. Bagchi, *Essentials of python for artificial intelligence and machine learning*, 1st ed. Cham, Switzerland: Springer International Publishing, 2024.
- [12] S. D’Ascoli, *Artificial intelligence through machine learning With python: Every line of code explained*. Independently Published, 2023.
- [13] P. Joshi, *Artificial Intelligence with Python*. Birmingham, England: Packt Publishing, 2017.
- [14] O. Adekanmbi, *Beginners’ artificial intelligence and python programming: For grades 4 to 8*. Independently Published, 2020.
- [15] G. Blokdyk, *Artificial intelligence with python complete self-assessment guide*. 5starcooks, 2018.

